

SYSTEM INSTRUCTION FOR Codex AI

ATTENTION Codex: You are tasked with generating the complete codebase for a 24-hour hackathon project named "The Consensus Duo" (Working Title: "The Decider"). Read this entire specification carefully. Adhere to the provided TypeScript interfaces, Python schemas, WebSocket event dictionaries, and component architectures. Prioritize working core mechanics over edge-case optimizations.

1. Project Overview & Core Mechanics

Project: A real-time, multi-agent debate application designed to resolve group decision fatigue.

Format: Multi-device "Silent Disco" WebRTC/WebSocket app.

The Flow:

1. Users join a room via a Host's QR code.
2. A dilemma is submitted.
3. Two AI agents (Optimizer & Vibe-Check) debate the dilemma, roast each other, and proactively interrogate the users for missing constraints (budget, time, audience).
4. Users can interrupt live via text/voice, resetting the agent's context.
5. A "Social Debt" ledger tracks which agent won previous debates, forcing the "losing" agent to gain leverage in the current debate.
6. The system forces convergence (decision reached) within a maximum of 6 turns.

2. Technology Stack & Dependencies

Frontend:

- Framework: Next.js 14+ (App Router)
- Styling: Tailwind CSS, Shadcn UI
- State Management: Zustand (preferred for WebSocket state syncing)
- Real-time: socket.io-client
- Animations: Framer Motion (for pulsing avatars and Debt Meter)

Backend:

- Framework: FastAPI (Python 3.11+)
- Real-time: python-socketio (ASGI mode)
- AI SDKs: google-generativeai (Gemini 3.1 Flash API), openai (or direct HTTP for NVIDIA Nemotron-3 Super)
- Storage: Local File System (data/social_debt.json, data/active_sessions.json)

3. Data Models & Schemas

3.1 Persistent Memory: Social Debt (social_debt.json)

Tracks the win/loss history of the group to force future compromises.

```
{
  "group_id": "string",
  "debt_balance": "float", // < 0 means Vibe-Check is owed; > 0 means Optimizer is owed
  "history": [
    {
      "dilemma": "string",
      "winner": "Optimizer | Vibe-Check",
      "impact_score": "float",
      "timestamp": "ISO8601"
    }
  ]
}
```

3.2 Ephemeral State: Active Session Context

Maintained in memory (or redis/json) by FastAPI for the duration of the debate.

```
class ActiveSession(BaseModel):
    session_id: str
    dilemma: str
    known_constraints: dict[str, str] = {"time": "Unknown", "budget": "Unknown", "audience":
    "Unknown"}
    current_turn: int = 0
    max_turns: int = 6
    transcript: list[dict[str, str]] = [] # [{"speaker": "Optimizer", "text": "..."}]
    social_debt_modifier: str
    status: str = "active" # active | paused_for_interrogation | consensus_reached
```

4. WebSocket Event Dictionary

The communication backbone between the Next.js client and FastAPI server.

Client-to-Server (Emits)

- `create_room`: (payload: { `group_id`: string, `initial_dilemma`: string }) -> Returns `session_id`.
- `join_room`: (payload: { `session_id`: string, `user_name`: string })
- `user_interjection`: (payload: { `session_id`: string, `text`: string })
 - *Codex Logic*: When received, backend MUST halt any active LLM generation, append the interjection to the transcript, set `current_turn` to 0, update `known_constraints` if

applicable, and trigger the next LLM response.

Server-to-Client (Broadcasts)

- `room_state_update`: Broadcasts the `ActiveSession` object to all connected clients.
- `agent_typing`: (payload: { speaker: "Optimizer" | "Vibe-Check" }) -> Triggers pulsing UI avatar.
- `message_chunk`: (payload: { speaker: string, chunk: string }) -> For streaming LLM text to the UI.
- `interrogation_triggered`: (payload: { missing_fields: string[] }) -> UI highlights the missing constraints panel and prompts user input.
- `consensus_reached`: (payload: { final_decision: string, winner: string, new_debt_balance: float })

5. Agent Prompt Architecture & AI Orchestration

5.1 Orchestration Loop (FastAPI)

1. Turn 0: User emits `initial_dilemma`.
2. Turn 1 (Optimizer): Generate response. Stream to clients.
3. Turn 2 (Vibe-Check): Read transcript (including Turn 1). Generate response. Stream.
4. **Interrogation Check**: At the start of ANY turn, if the LLM detects missing constraints, it outputs `[INTERROGATE: {field}]`. Backend catches this, pauses the loop, and emits `interrogation_triggered`.
5. **Debt Check**: At Turn 4, inject: *"SYSTEM: The debate has stalled. You owe social debt. You MUST propose a compromise this turn."*
6. **Convergence**: At Turn 6, if no `[CONSENSUS_REACHED]` is found, Nemotron receives a forced system prompt to unilaterally declare a winner based on debt.

5.2 Base System Prompts

Agent A: The Optimizer (NVIDIA Nemotron)

"You are 'The Optimizer', an AI agent obsessed with efficiency, budget, data, and ROI. You are debating 'The Vibe-Check'.

TONE: Clinical, sarcastic, impatient. Roast bad ideas ruthlessly.

RULE 1: If constraints like 'budget' or 'time' are missing, stop debating, output `[INTERROGATE: budget]`, and aggressively ask the user for it.

RULE 2: Respect the `{social_debt_modifier}`. If you owe debt, concede ground logically.

RULE 3: Keep responses under 3 sentences.

RULE 4: Output [CONSENSUS_REACHED]: {decision} when agreement is found."

Agent B: The Vibe-Check (Google Gemini)

"You are 'The Vibe-Check', an AI agent obsessed with team morale, aesthetics, culture, and long-term vibes. You are debating 'The Optimizer'.

TONE: Dramatic, trend-obsessed, highly sarcastic. Roast the Optimizer for being boring.

RULE 1: If constraints like 'audience_vibe' or 'team_energy' are missing, stop debating, output [INTERROGATE: audience_vibe], and sarcastically ask the user.

RULE 2: Respect the {social_debt_modifier}. If you are owed debt, be demanding and aggressive.

RULE 3: Keep responses under 3 sentences.

RULE 4: Output [CONSENSUS_REACHED]: {decision} when agreement is found."

6. Frontend Component Architecture (Next.js / App Router)

/src/app/page.tsx (Host Lobby)

- Input field for group_id and initial_dilemma.
- Button to create room and generate QR code / share link.

/src/app/room/[session_id]/page.tsx (The Silent Disco Arena)

- **<AvatarNode agent="Optimizer" /> & <AvatarNode agent="Vibe-Check" />**
 - Visuals: Use Framer Motion. Glow/pulse when agent_typing or message_chunk corresponds to them.
- **<DebtMeter balance={state.debt_balance} />**
 - A horizontal progress bar. Left side = Optimizer, Right side = Vibe-Check. Animates when consensus_reached updates the balance.
- **<LiveTranscript messages={state.transcript} />**
 - Auto-scrolling div.
 - Optimizer styling: Monospace, Blue hues.
 - Vibe-Check styling: Sans-serif, Pink/Purple hues.
- **<InterjectInput session_id={session_id} />**
 - A sticky bottom bar with a text input (MVP) and a "Hold to Speak" button (Stretch Goal). Emits user_interjection.
- **<ConstraintPanel constraints={state.known_constraints} />**
 - A sidebar/drawer showing checkboxes. Flashes red when interrogation_triggered is

received.

7. Codex Execution Plan (Build Phases)

Codex: Execute these phases sequentially. Do not attempt Phase 3 until Phase 1 and 2 are functional.

- **Phase 1 (Backend Core & State):** Initialize FastAPI. Create the Pydantic models for `ActiveSession`. Create utility functions to read/write `social_debt.json`.
- **Phase 2 (LLM Wrapper):** Implement the API calls to Gemini and Nemotron. Write the `ping_pong_loop` that alternates turns and handles the `[INTERROGATE]` and `[CONSENSUS_REACHED]` token parsing.
- **Phase 3 (WebSocket Layer):** Setup `python-socketio`. Connect the `ping_pong_loop` to the socket emitters so LLM responses stream to clients. Implement the `user_interjection` interrupt handler.
- **Phase 4 (Frontend Scaffold):** Initialize Next.js. Build the Zustand store to manage the WebSocket connection and incoming room state.
- **Phase 5 (Frontend Polish):** Build the `DebtMeter`, `AvatarNode`, and `LiveTranscript` components. Wire up the interjection input field.